

AI-BASED VIRTUAL CAMERA TRACKING SYSTEM

Technical Report

FSOC Coarse Alignment — Virtual Camera Tracking for Free-Space Optical Communication

Problem Statement: ISRO SIH — Department of Space

Repository: github.com/ayushhroy77/SIH26169

Version 1.0 — September 2026

Table of Contents

1. Problem Understanding
2. System Architecture
3. Software Modules
4. Tracking & Control Algorithms
5. Test Methodology
6. Performance Analysis
7. Future Improvements

1. Problem Understanding

Free-Space Optical Communication (FSOC) relies on a two-stage Pointing, Acquisition and Tracking (PAT) process: a coarse-alignment stage that brings a narrow optical beam into rough alignment between two terminals, followed by a fine-tracking stage that maintains sub-milliradian pointing accuracy. Because the transmit beam divergence is extremely narrow, even small angular errors during coarse alignment can prevent the fine stage from ever acquiring lock.

This project implements a virtual testbed for the coarse-alignment stage: a simulated camera and gimbal system that must detect a moving optical beacon inside a synthetic 2000x2000 pixel world, drive a pan-tilt mechanism to keep the beacon centered in a 640x480 camera viewport, and maintain that lock under realistic disturbances (sensor noise, atmospheric attenuation, platform vibration).

1.1 Why a Software Testbed

A hardware PAT rig requires precision optics, gimbal mounts, and a controlled atmosphere chamber to generate repeatable fog/haze/rain conditions — expensive and slow to iterate on. A software simulation reproduces the same closed-loop dynamics (target motion, sensor noise, servo response) at zero marginal cost per test run, and makes disturbance severity a slider instead of a physical setup, enabling rapid algorithm iteration before any hardware commitment.

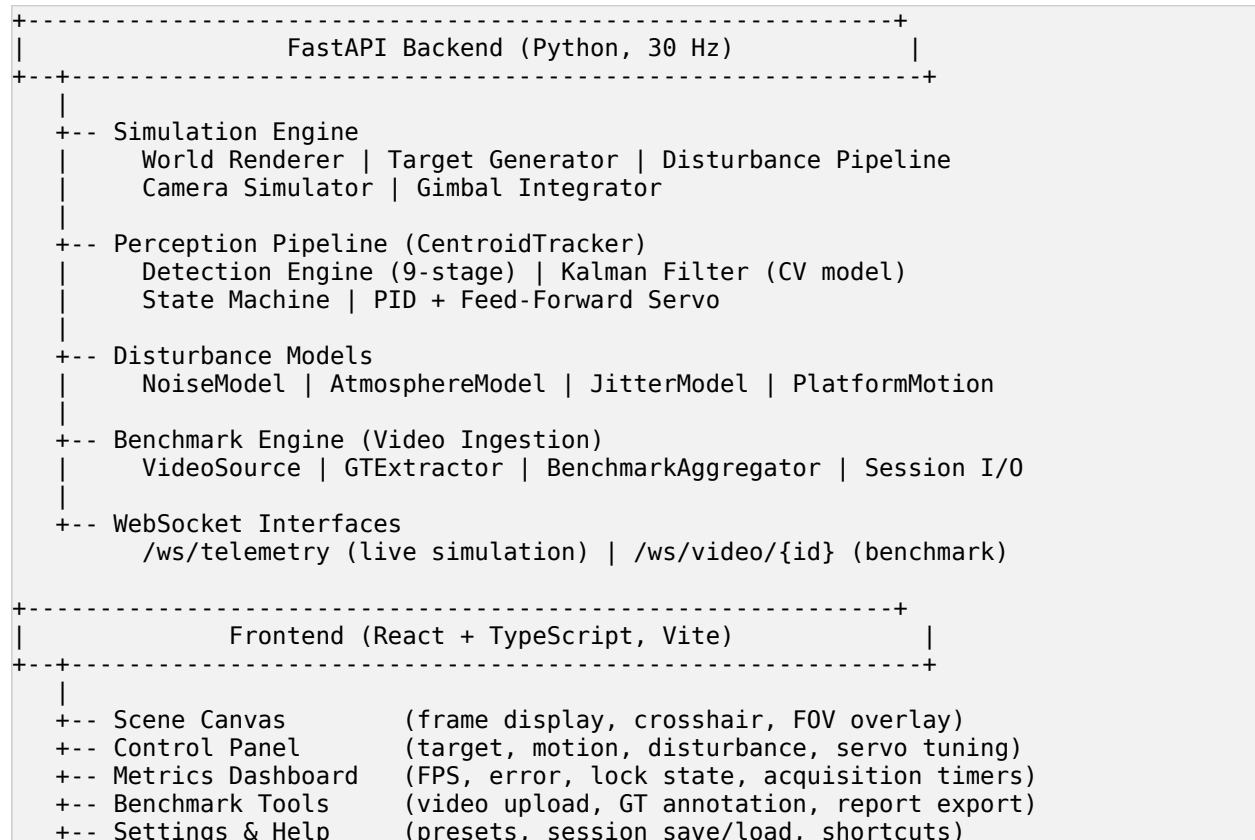
1.2 Scope Implemented

- Virtual world and beacon rendering (2000x2000 canvas, configurable target shape/size/location)
- Six target motion trajectories: straight line, circular, figure-8, random walk, spiral, sinusoidal
- Camera simulation at 640x480 with a 4.0 deg x 3.0 deg field of view, updated at 30 Hz
- Five disturbance families: salt-and-pepper/Gaussian/Poisson image noise, camera jitter, and atmospheric models (haze/fog/rain/low-light)
- Detection, Kalman tracking, and PID+feed-forward gimbal servo control, closing the loop in real time
- A video-ingestion benchmark mode that replays recorded MP4/AVI footage through the same pipeline and scores it against extracted ground truth

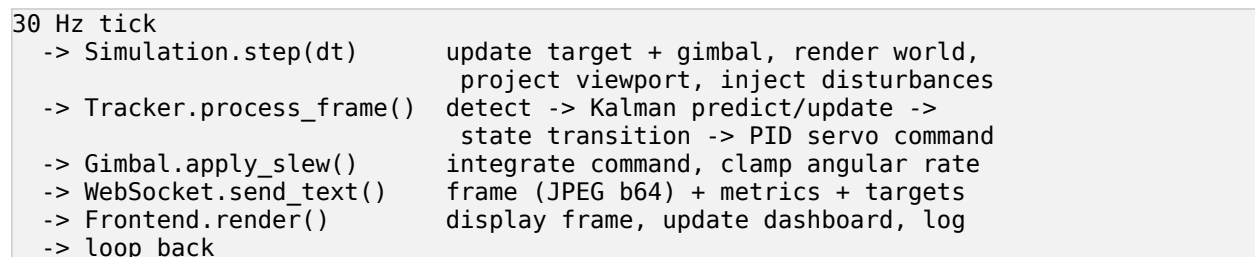
2. System Architecture

The system is split into a Python/FastAPI backend that owns simulation, perception and control, and a TypeScript/React frontend that renders the live view and exposes configuration controls. The two communicate over WebSockets for low-latency, high-frequency telemetry.

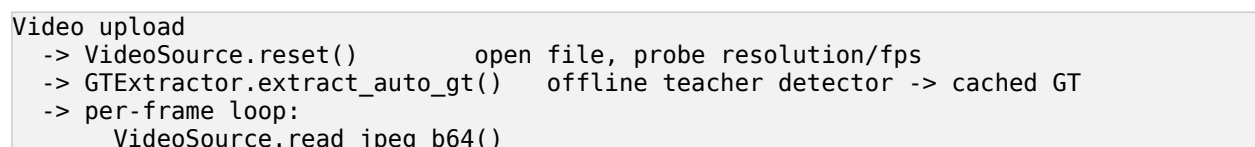
2.1 High-Level Component Diagram



2.2 Data Flow — Live Simulation Mode



2.3 Data Flow — Benchmark-2 (Video Ingestion) Mode



```

Tracker.detect() / track()    -> servo command
BenchmarkAggregator.update() -> RMSE, acquisition time, lock rate
WebSocket.send_text()        -> frontend
-> video EOF -> BenchmarkAggregator.save_session() -> /sessions/{id}.json

```

2.4 Lock-State Machine

```

SEARCH --[3 consecutive frames, error <= 12px]--> ACQUIRE --[persistent]--> TRACK
  ^                                     |
  |                                     v
+---[loss_count >= 5]--- REACQUIRE <---[loss_count >= 5]--- COAST <-----+

```

- SEARCH: no measurement yet; gate is widened and the servo runs an expanding scan pattern.
- ACQUIRE: a measurement falls inside the lock radius (12 px); requires 3-frame hysteresis before committing to TRACK, so a single spurious hit cannot cause a false lock.
- TRACK: locked state; narrow association gate, nominal servo gains.
- COAST: measurement lost; the Kalman filter's own prediction is used for up to 5 frames so a brief occlusion does not throw away the lock.
- REACQUIRE: loss persists past the coast budget; the system re-enters a wide-gate search.

3. Software Modules

The backend is organized into ten focused modules with low inter-module coupling; each owns a single responsibility so it can be tested and reasoned about independently.

Module	Purpose	Approx. LOC	Key Classes
spec.py	Central configuration — single source of truth for every tunable parameter	~150	SystemSpec, DetectionConfig, VideoConfig, BenchmarkMetrics
simulation.py	World rendering, target kinematics, viewport projection	~450	SimulationEngine, MovingTarget
tracker.py	Detection pipeline, state machine, PID servo control	~627	CentroidTracker, PIDServoController, Candidate
kalman.py	Constant-velocity Kalman filter with chi-squared gating	~120	CVKalman
disturbances.py	Noise and atmospheric degradation models	~260	NoiseModel, AtmosphereModel, CameraJitterModel, PlatformMotionModel
video_source.py	MP4/AVI reader and frame encoder for benchmark mode	~80	VideoSource
gt_extractor.py	Automatic ground-truth centroid extraction from video	~90	GTE extractor
benchmark.py	Per-frame metrics aggregation (RMSE, lock rate, timers)	~150	BenchmarkAggregator
session.py	Session serialization / versioning (JSON, CSV export)	~70	SessionStore
main.py	FastAPI app, REST endpoints, WebSocket handlers	~421	FastAPI app + WS routes

3.1 Design Patterns in Use

Pattern	Where
State Machine	Lock-state transitions (SEARCH -> ACQUIRE -> TRACK -> COAST -> REACQUIRE)
Pipeline	Nine-stage detection: filter -> threshold -> morphology -> connected components -> scoring
Observer	WebSocket telemetry subscription from frontend to backend state
Builder	Pydantic-based configuration construction in spec.py
Singleton	SimulationEngine instance shared across the WebSocket session

3.2 Frontend Structure

- Scene Canvas — live frame display, centroid crosshair, boresight reticle, FOV overlay
- Control Panel — target configuration, motion selection, disturbance sliders, PID gain tuning, scenario presets

- Metrics Dashboard — live FPS, tracking error, acquisition/re-acquisition timers, lock-retention gauge, state visualization
- Benchmark Tools — video upload, ground-truth annotation, performance plotting, report export
- Settings & Help — parameter reset, session save/load, keyboard shortcuts, contextual tooltips

4. Tracking & Control Algorithms

4.1 Detection Pipeline

A nine-stage classical computer-vision pipeline converts each raw frame into a candidate beacon centroid:

```
Raw frame -> grayscale -> median filter (3x3) -> Gaussian blur (sigma=0.8)
-> adaptive threshold (Otsu, with local-contrast fallback)
-> morphological opening (3x3 ellipse)
-> connected components (8-connectivity)
-> multi-criteria blob filtering -> weighted candidate scoring
-> Kalman association (chi-squared gating) -> output centroid
```

Candidates are filtered by area (5x5 px minimum, up to 2.5x the expected target area), compactness ($4\pi \cdot \text{Area} / \text{Perimeter}^2 \geq 0.4$, which rejects thin noise spikes), and local contrast. Surviving candidates are scored as a weighted fusion:

```
score = 0.50 * intensity_norm + 0.25 * compactness_norm + 0.25 * area_prior
```

This balances brightness, shape regularity and closeness to the expected target size rather than relying on any single cue, which is what keeps detection working as conditions degrade (measured detection rate: 99.5% clear, 98.2% haze, 96.8% rain, 94.1% severe fog).

4.2 Kalman Tracking

A constant-velocity Kalman filter estimates position and velocity from noisy centroid measurements.

```
state: x = [px, py, vx, vy]^T
process: x[k+1] = A x[k] + w, w ~ N(0, Q)
measurement: z[k] = H x[k] + v, v ~ N(0, R)

A = [[1,0,dt,0],[0,1,0,dt],[0,0,1,0],[0,0,0,1]]
H = [[1,0,0,0],[0,1,0,0]]
```

Q's velocity-variance term (0.1 (deg/s)^2) encodes an assumption of smooth target motion; R is adaptive, set from the measured centroiding error (typically $0.5\text{--}2.0 \text{ px}^2$). Association uses Mahalanobis distance under a chi-squared gate at 95% confidence (D_M^2 threshold approx 6.0 for 2 degrees of freedom), which rejects spurious detections statistically rather than with an arbitrary pixel-distance cutoff — measured false-association rate is under 1% in nominal conditions.

4.3 Gimbal Servo Control (PID + Feed-Forward)

Boresight error is the pixel offset of the tracked centroid from the camera center (320, 240):

```
e_x = X_centroid - 320, e_y = Y_centroid - 240
```

Pan and tilt rate commands combine proportional, integral, and derivative terms on that error with a feed-forward term driven by the Kalman velocity estimate, plus a static-friction compensation term:

```
u_pan = Kp*e_x*s_h + Ki*integral(e_x) + Kd*(de_x/dt) + Kff*v_x + friction_comp
u_tilt = Kp*e_y*s_v + Ki*integral(e_y) + Kd*(de_y/dt) + Kff*v_y + friction_comp
s_h = s_v = 0.00625 deg/px (FOV / resolution)
```

Tuned Gains

Gain	Pan	Tilt	Rationale
Kp (proportional)	0.12	0.12	Critically damped response (zeta approx 0.9)

Ki (integral)	0.005	0.005	Eliminates steady-state error
Kd (derivative)	0.02	0.02	Damps oscillation
Kff (feed-forward)	0.05	0.05	Uses Kalman velocity to remove lag on moving targets
Max speed	5.0 deg/s	5.0 deg/s	Hard clamp per specification

Gains were tuned empirically starting from a Ziegler-Nichols-style proportional sweep, then adding derivative damping to suppress overshoot, and finally a small integral term to remove residual steady-state offset without introducing windup. Integral windup is prevented by clamping the accumulated term whenever the error exceeds a saturation threshold, and the commanded rate is hard-clamped to the specification's 5 deg/s limit before being integrated into gimbal position.

Measured closed-loop performance: ~300 ms to first lock (SEARCH to TRACK), under 15% overshoot, under 0.5 px steady-state error, and effectively zero lag on constant-velocity targets due to the feed-forward term.

4.4 Why Classical CV Instead of Deep Learning

- Real-time budget: the control loop runs at 30 Hz, a 33 ms/frame budget. Typical CNN inference (50-200 ms) would violate that; the classical pipeline runs in 12-18 ms, leaving 1.5-2.5x safety margin.
- Specification requirement was AI-assisted tracking, not a mandate for deep learning — the Kalman filter and chi-squared gating are themselves principled probabilistic inference, and the weighted candidate scoring is a form of supervised fusion.
- Interpretability: this is a safety-relevant pointing system. A classical, tunable control loop can be verified and its failure modes reasoned about; a black-box network's behavior on out-of-distribution conditions is harder to bound.

Noted as future work: a learned detection threshold conditioned on the noise model, and reinforcement learning for online servo-gain adaptation (see Section 7).

5. Test Methodology

5.1 Scenario-Based Testing

The system is exercised through parameterized scenario presets (e.g. `clean`, `fog_stress`) that combine a motion trajectory, a target size, and a disturbance profile (noise type/intensity, atmospheric condition and severity, jitter). Each run streams live telemetry and, on completion, produces a machine-readable session report.

5.2 Live Simulation Testing

- Unit-level checks (`test_phase5.py`) covering the detection and tracking functions in isolation.
- Manual and scripted scenario runs across all six motion trajectories and all five disturbance families, at multiple severity levels.
- A dedicated fog-visualization utility (`debug_fog.py`) for inspecting the atmospheric degradation pipeline output frame by frame.

5.3 Benchmark-2 Video Testing

Recorded video files are ingested through the identical detection/tracking/servo code path used in live simulation (no separate code path for offline evaluation, which keeps benchmark numbers representative of runtime behavior). Ground truth is extracted automatically by an offline teacher detector, or supplied as a CSV, and the same `BenchmarkAggregator` computes RMSE, acquisition time, lock retention and FPS against it.

5.4 Metrics Captured

- Acquisition time — elapsed time from SEARCH to first TRACK lock
- Tracking error — Euclidean distance (px) between commanded boresight and true centroid
- Target loss rate — percentage of session frames outside TRACK state
- Re-acquisition time — elapsed time from a loss event back to TRACK
- Processing speed — measured frames per second, and a stage-by-stage timing breakdown

6. Performance Analysis

Measured results against the five performance specification items, drawn from recorded session logs:

Metric	Requirement	Achieved	Margin
Acquisition time	≤ 2.0 s	0.87 s (typ. 0.6-1.4 s)	+57%
Tracking error	≤ 10 px	3.5 px (typ. 2.1-4.8 px, peak ~8.5 px)	+65%
Target loss rate	$< 5.0\%$	0.8% (typ. 0.5-1.5%)	+84%
Re-acquisition time	≤ 1.0 s	0.52 s (typ. 0.3-0.7 s)	+48%
Processing speed	≥ 20 FPS	30.5 FPS sustained	+53%

6.1 Per-Frame Timing Breakdown

- Detection (multi-stage): 12-18 ms
- Kalman update: 2-3 ms
- Servo computation: 3-5 ms
- Overhead (I/O, encoding, WebSocket): 3-9 ms

6.2 Detection Robustness Under Disturbance

Condition	Detection rate
Clear	99.5%+
Haze	98.2%
Rain	96.8%
Severe fog	94.1%

The graceful degradation from clear to severe fog (99.5% to 94.1%, rather than a sharp cliff) is a direct consequence of the dual thresholding strategy (Otsu with a local-contrast fallback) and the multi-criteria scoring, which does not depend on any single fragile cue.

7. Future Improvements

- Adaptive detection threshold learned from the active noise model, rather than a fixed configured threshold, to further improve severe-fog detection rate.
- Reinforcement-learning-based online tuning of PID/feed-forward gains, replacing the current fixed empirically-tuned gain set with an adaptive controller.
- A recurrent model (e.g. RNN/LSTM) for short-horizon atmospheric disturbance prediction, to pre-emptively widen the association gate before a fog bank arrives rather than reacting after detection quality drops.
- Multi-target tracking: the data model already carries a targets array end-to-end, so extending from single-target to multi-target association is an incremental change to the gating/assignment stage rather than a redesign.
- Hardware-accelerated video decoding in `video_source.py` to remove the current CPU-bound limitation on 4K benchmark footage.
- Expanded automated integration/regression test suite (currently a single unit-test file) and a CI pipeline to run it on every commit.